

## A. 彩虹遊行

Problem ID: Rainbow\_Parade

Time Limit: 1s

Memory Limit: 512MiB

家豪（Charhao）身為台南一中的三好學生，平時最喜歡翹課出來參加彩虹遊行。顧名思義，彩虹遊行指的是一大群彩虹小馬愛好者聚在一起的盛大活動。

今天的彩虹遊行在一座神奇的島嶼上舉辦，島上有  $n$  個彩虹小馬景點，編號為  $1 \sim n$ 。島嶼的入口位於 1 號景點，而開幕式場地則在  $n$  號景點。

家豪對這個島嶼一無所知，一個人走一定會迷路，所以他需要熱心助人的彩虹小馬學長 tw87 帶路。島上共有  $m$  條單向道路，第  $i$  條道路從景點  $a_i$  連向景點  $b_i$ 。

由於 tw87 趕著去參加「彩虹小馬聖經研討會」，無法親自帶路，於是他決定畫一張地圖給家豪。地圖的繪製規則如下：

- 除了開幕式景點  $n$  之外，每個景點都必須**恰好選擇**一條從該點出發的單向道路，寫下一個該去的景點。
- 為了避免地圖標示混亂，除了入口 1 號景點外，其餘每個景點在地圖上**恰好只能被一個景點指引**。
- 家豪必須能順利從入口 1 號景點沿著指引走到開幕式  $n$  號景點。但為了滿足 tw87 的惡趣味，對於**不在這條主線路徑上**的其他景點，tw87 刻意將它們設計成**若家豪不小心闖入，將會陷入無限繞圈圈的死循環中**。

每條道路  $i$  都有一個「奇怪值」 $d_i$ 。只要 tw87 在地圖上記錄了該條道路，該道路的奇怪值就會累加到總和中。

tw87 想在滿足自己惡趣味的前提下，盡可能最小化總奇怪值，以免被可愛的學弟討厭。身為一中的彩虹新秀，請你幫 tw87 計算出最小可能的總奇怪值為多少？

### — 輸入 —

第一行輸入二正整數  $n$   $m$

接下來  $m$  行，輸入二正整數與一非負整數  $a_i, b_i, d_i$

### — 輸出 —

輸出一個整數，代表最小可能的總奇怪值。

若無法構造出滿足條件的地圖，則輸出 -1。

### — 輸入限制 —

- $1 \leq n \leq 2000$
- $1 \leq m \leq \min(10^4, n \times (n - 1))$
- $1 \leq a_i, b_i \leq n, a_i \neq b_i$
- $0 \leq d_i \leq 10^5$

### — 子任務 —

| 編號 | 分數 | 額外限制                                                                               |
|----|----|------------------------------------------------------------------------------------|
| 1  | 0  | 範例輸入輸出                                                                             |
| 2  | 8  | $1 \leq n \leq 8, 1 \leq m \leq \min(30, n \times (n - 1)), 0 \leq d_i \leq 100$   |
| 3  | 17 | $1 \leq n \leq 18, 1 \leq m \leq \min(200, n \times (n - 1)), 0 \leq d_i \leq 100$ |
| 4  | 22 | $1 \leq n \leq 1000, 1 \leq m \leq \min(5000, n \times (n - 1)), d_i \leq 1$       |
| 5  | 24 | $1 \leq n \leq 200, 1 \leq m \leq \min(1000, n \times (n - 1)), d_i \leq 10^5$     |
| 6  | 29 | 無額外限制                                                                              |

— 範例輸入 —

— 範例輸出 —

## D. Merge, Rebase, Squash!

Problem ID: Git

Time Limit: 2.5s

Memory Limit: 512MiB



Force push 1



Force push 2

Figure 1: 不要學 DB force push (他現在已經不會了)

tobiichi3227 是 TOJ 的維護者，TOJ 整個程式碼由  $H$  個獨立的 **hunk** 組成，每個 hunk 的狀態一開始都是 0，在 v2.0 分支上面。

有天 wonderhoi 想給 TOJ 貢獻一個酷炫的功能，於是 wonderhoi 很高興的開啟了一個新的分支 feat/fancy 並在上面建立了  $N$  個 commit。

wonderhoi 想讓功能進到 TOJ 的 v2.0，於是請 tobiichi3227 來 review，tobiichi3227 覺得  $N$  個 commit 太多了，應該用 squash 把 commit 減少到只有  $K$  個。

當 squash 完成後，會將 feat/fancy rebase 到 v2.0，wonderhoi 發現會有 conflict，所以他想要最小化解決 conflict 的成本，但 wonderhoi 要去比地奧 (IESO) 沒空了，於是交給在看题目的你。



Figure 2: Branch、squash、rebase 與 conflict resolution 的執行過程

若使用支援 PDF 附件的 Firefox，可  
分鏡。

；Chrome 與 Edge 請直接閱讀上方

## — Branch 狀態 —

v2.0 branch 上第  $h$  個 hunk 的最終內容為  $U_h$ 。

feat/fancy branch 上共有  $N$  個 commits。第  $i$  個 commit 會將第  $x_i$  個 hunk 的內容改成  $v_i$ 。

令  $S_i[h]$  表示執行完 feat/fancy branch 的前  $i$  個 commits 後，第  $h$  個 hunk 的內容。

初始時： $S_0[h] = 0$

第  $i$  個 commit 執行後： $S_i[x_i] = v_i$

其他 hunk 的內容不變。

## – Squash –

原本的  $N$  個 commits 依照順序排列：

commit 1, commit 2, ..., commit  $N$

你需要在這些 commits 之間放置  $K - 1$  個切點，將它們分成恰好  $K$  組。

每一組中的所有 commits 會被合併成一個新的 **squash commit**。

例如，假設共有 7 個 commits，且要分成 3 組，可以選擇：

```
[commit 1, commit 2]
[commit 3, commit 4, commit 5]
[commit 6, commit 7]
```

也可以簡寫成：

```
[1, 2] [3, 4, 5] [6, 7]
```

這代表：- 第一個 squash commit 包含原本的 commits 1 到 2 - 第二個 squash commit 包含原本的 commits 3 到 5 - 第三個 squash commit 包含原本的 commits 6 到 7

為了描述一般情況，令： $0 = p_0 < p_1 < p_2 < \dots < p_K = N$

其中  $p_g$  表示第  $g$  組的最後一個 commit 編號。

因此，第  $g$  個 squash commit 包含： $p_{g-1} + 1, p_{g-1} + 2, \dots, p_g$

例如前面的分組：

```
[1, 2] [3, 4, 5] [6, 7]
```

對應： $p_0 = 0, p_1 = 2, p_2 = 5, p_3 = 7$

考慮第  $g$  個 squash commit，以及某個 hunk  $h$ 。

在這組 commits 開始以前，hunk  $h$  的內容是： $S_{p_{g-1}}[h]$

執行完這組中的所有 commits 後，hunk  $h$  的內容是： $S_{p_g}[h]$

因此，這個 squash commit 對 hunk  $h$  的效果為： $S_{p_{g-1}}[h] \rightarrow S_{p_g}[h]$

也就是只保留這一組 commits 執行前與執行後的差異，中間經過的狀態不會出現在 squash commit 中。

例如，某個 hunk 在一組 commits 中依序發生：

1 -> 3 -> 5 -> 2

那麼 squash 後只會留下：

1 -> 2

中間的 3 與 5 都不會出現在 squash commit 中。

若一組 commits 執行前後，某個 hunk 的內容相同，即： $S_{p_{g-1}}[h] = S_{p_g}[h]$

則這個 squash commit 不會修改該 hunk。

## – Rebase 與 Conflict –

完成 squash 後，tobiichi3227 會依照原本的順序，將這  $K$  個 squash commits 逐一套用到 v2.0 branch 上。

考慮其中一個 squash commit 對 hunk  $h$  的修改： $\text{old} \rightarrow \text{new}$

其中：- old：這個 squash commit 預期修改前的內容；- new：這個 squash commit 想要修改成的內容；- current：目前 v2.0 branch 上實際存在的內容。

例如： $\text{old} = 3$ ,  $\text{new} = 7$ ,  $\text{current} = 5$

代表這個 squash commit 原本是在內容為 3 的基礎上，將它改成 7，但目前 v2.0 branch 上的內容實際是 5。

### 沒有修改

若： $\text{old} = \text{new}$

代表這個 squash commit 執行前後，該 hunk 的內容沒有改變。

因此不需要進行任何操作，current 也不會改變。

### 情況一：目前內容正是預期的舊內容

若： $\text{current} = \text{old}$

代表目前內容與這個 patch 預期的起點相同，因此可以正常套用修改。

套用後： $\text{current} \leftarrow \text{new}$

例如：

```
patch:    3 -> 7
current: 3
```

可以正常套用，完成後 current 變成 7。

### 情況二：目前內容已經是目標內容

若： $\text{current} = \text{new}$

代表 v2.0 branch 上已經存在這個 squash commit 想要產生的結果。

因此不會發生 conflict，也不需要再次修改。

例如：

```
patch:    3 -> 7
current: 7
```

雖然目前內容不是 patch 預期的舊內容 3，但它已經等於目標內容 7，所以可以直接視為這項修改已經存在。

current 維持為 7。

### 情況三：目前內容既不是舊內容，也不是新內容

若： $\text{current} \neq \text{old}$

且： $\text{current} \neq \text{new}$

代表 v2.0 branch 上的內容與這個 squash commit 預期的內容不同，而且也不是它想要修改成的結果。

此時會發生 conflict，例如：

patch: 3 -> 7

current: 5

目前內容 5 既不是舊內容 3，也不是新內容 7，因此發生 conflict。

解決 hunk  $h$  的 conflict 需要支付  $w_h$  的成本。

解決 conflict 後，採用 squash commit 的修改結果： $\text{current} \leftarrow \text{new}$

在前面的例子中，解決後 current 會變成 7。

同一個 squash commit 可能同時修改多個 hunks。每個 hunk 是否發生 conflict 都要分別判斷，發生 conflict 的成本也要分別計算。

## — 輸入 —

第一行包含三個整數  $N$ ,  $H$ ,  $K$ , 分別表示：- feat/fancy branch 上的 commit 數量；- hunk 的數量；- squash 後必須保留的 commit 數量。

第二行包含  $H$  個整數  $U_h$  ( $1 \leq h \leq H$ )

其中  $U_h$  表示 v2.0 branch 上第  $h$  個 hunk 的最終內容。

第三行包含  $H$  個整數  $w_h$  ( $1 \leq h \leq H$ )

其中  $w_h$  表示解決第  $h$  個 hunk 的 conflict 所需的成本。

接下來  $N$  行，第  $i$  ( $1 \leq i \leq N$ ) 行包含兩個整數  $x_i$  與  $v_i$  表示第  $i$  個 commit 將第  $x_i$  個 hunk 的內容改成  $v_i$ 。

## — 輸出 —

輸出一個整數，表示最小的 conflict 解決總成本。

## — 輸入限制 —

- $1 \leq N, H \leq 200000$
- $1 \leq K \leq \min(N, 30)$
- $1 \leq U_h \leq 10^9$
- $1 \leq w_h \leq 10^9$
- $1 \leq x_i \leq H$
- $1 \leq v_i \leq 10^9$
- 對於每個 commit  $i$  :  $v_i \neq S_{i-1}[x_i]$
- 保證每個 commit 都會實際改變該 hunk 的內容，且 feat/fancy branch 不會將任何 hunk 改回 0。

### — 子任務 —

| 編號 | 分數 | 額外限制             |
|----|----|------------------|
| 1  | 0  | 範例輸入輸出           |
| 2  | 8  | $N, H \leq 15$   |
| 3  | 8  | $K = 1$          |
| 4  | 8  | $K = N$          |
| 5  | 10 | 每個 hunk 最多只被修改一次 |
| 6  | 26 | $N \leq 1000$    |
| 7  | 40 | 無限制              |

### — 範例輸入 —

```
5 2 2
1 4
5 3
1 1
2 2
1 2
2 4
1 3
```

### — 範例輸出 —

```
0
```

### — 範例說明 —

feat/fancy branch 上兩個 hunk 的狀態依序如下：

```
commit 0: (0, 0)
commit 1: (1, 0)
commit 2: (1, 2)
commit 3: (2, 2)
commit 4: (2, 4)
commit 5: (3, 4)
```

可以將 commits 分成：

[1] [2, 3, 4, 5]

第一個 squash commit 將 hunk 1 :

0 -> 1

套用到 v2.0 branch 時，hunk 1 的目前內容已經是 1，因此不會發生 conflict。

第二個 squash commit 的效果為：

hunk 1 : 1 -> 3

hunk 2 : 0 -> 4

此時 hunk 1 的目前內容為 1，可以正常套用。

hunk 2 的目前內容已經是 4，因此也不會發生 conflict。

總成本為 0。

## — 範例輸入 —

```
2 1 1
1
7
1 1
1 2
```

## — 範例輸出 —

```
7
```

## — 範例說明 —

因為  $K = 1$ ，兩個 commits 必須全部 squash 在一起。  
Squash 後的效果為：

```
0 -> 2
```

但 v2.0 branch 上該 hunk 的內容為 1。

目前內容既不是 patch 的舊內容 0，也不是新內容 2，因此發生 conflict，成本為 7。

## — 範例輸入 —

```
2 1 2
1
7
1 1
1 2
```

## — 範例輸出 —

```
0
```

## — 範例說明 —

因為  $K = 2$ ，兩個 commits 分別套用。  
第一個 commit 的效果為：

```
0 -> 1
```

而 v2.0 branch 上目前的內容已經是 1，因此不會發生 conflict。  
第二個 commit 的效果為：

1 -> 2

此時目前內容為 1，可以正常套用。  
因此總成本為 0。

— **Note** —

## G. Performance Analysis

Problem ID: Compiler

Time Limit: 1s

Memory Limit: 512MiB

tobiichi3227 跟 yushiuan9499 最近在開發 NCC，tobiichi3227 發現它程式執行太慢了，所以想給 NCC 加上一個功能，統計每一條 control-flow edge 會執行幾次。

編譯器將程式用控制流圖 (Control Flow Graph，後面簡稱 CFG) 表示，CFG 上面會有  $N$  個基本塊 (Basic Block，後面簡稱 BB)，每個 BB 會有幾個有向邊連到其他 BB，代表從  $BB_i$  可以走到  $BB_j$ ，總共有  $M$  條有向邊。

程式會從入口 1 開始，最後從出口  $N$  離開，總共執行  $R$  次程式。

對於每條邊，編譯器在它所連接的 BB 上插入性能計數器 ( $BB_i$  或  $BB_j$  都可以，編譯器只會選一個 BB 插入)，因為測量執行次數會造成額外的運算，所以插入每個計數器需要成本  $c_e$ ，執行結束後就能知道該邊通過的次數。

對於沒有裝計數器的邊，可以用 flow conservation 推導計算得知：

- 對於普通節點:  $\sum_{e \in \text{in}(v)} f_e = \sum_{e \in \text{out}(v)} f_e \ (v \neq 1, N)$
- 對於入口:  $\sum_{e \in \text{out}(1)} f_e - \sum_{e \in \text{in}(v)} f_e = R$
- 對於出口:  $\sum_{e \in \text{in}(N)} f_e - \sum_{e \in \text{out}(N)} f_e = R$

求最小安裝成本，使得執行結束後能夠**唯一確定每一條邊通過的次數**。

### — 輸入 —

第一行會有三個整數  $N, M, R$ ，代表 BB 的數量、邊的數量與執行次數

接下來  $M$  行會有三個整數  $u, v, C$ ，代表  $BB_u$  可以走到  $BB_v$ ，且安裝計數器成本為  $C$

### — 輸出 —

最小安裝成本

### — 輸入限制 —

- $1 \leq N \leq 10^5$
- $1 \leq M \leq 2 \times 10^5$
- $2M \leq R \leq 10^{18}$
- $1 \leq u, v \leq N$
- $1 \leq C_e \leq 10^9$
- 允許 self-loop 與重邊
- 保證所有 BB 一定有一個 1 到  $N$  的路徑通過

### — 子任務 —

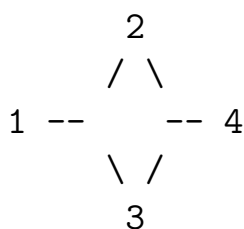
| 編號 | 分數 | 額外限制                                           |
|----|----|------------------------------------------------|
| 1  | 0  | 範例輸入輸出                                         |
| 2  | 5  | $M = N - 1$ , 且第 $i$ 條邊為 $i \rightarrow i + 1$ |
| 3  | 10 | $C_e = 1$                                      |
| 4  | 15 | $M \leq 20$                                    |
| 5  | 15 | $M = N$ 且無 self-loop 或重複連接                     |
| 6  | 20 | 每個點雙連通分量皆為單邊分量，或其中每個頂點的分量內度數皆為 2               |
| 7  | 35 | 無額外限制                                          |

— 範例輸入 —

— 範例輸出 —

— 範例解釋 —

這張 CFG 的結構如下：



程式總共執行 8 次。每次執行會選擇下列其中一條路線：

$1 \rightarrow 2 \rightarrow 4$  或  $1 \rightarrow 3 \rightarrow 4$

可以花費 2，在邊  $1 \rightarrow 3$  上安裝計數器。

假設程式執行完畢後，計數器顯示邊  $1 \rightarrow 3$  總共被通過 3 次。因為程式總共執行 8 次，所以邊  $1 \rightarrow 2$  必定被通過  $8 - 3 = 5$  次。

每次進入  $BB_2$  後都會沿著邊  $2 \rightarrow 4$  離開，因此邊  $2 \rightarrow 4$  也被通過 5 次。

同理，邊  $3 \rightarrow 4$  被通過 3 次。

因此，只要在邊  $1 \rightarrow 3$  上安裝一個計數器，就能確定所有邊的通過次數。

若不安裝任何計數器，則無法判斷 8 次執行中分別有多少次經過  $BB_2$  或  $BB_3$ 。因此至少需要安裝一個計數器。

所有邊中最低的安裝成本為 2，所以最小總費用為 2。

— 範例輸入 —

— 範例輸出 —

— 範例輸入 —

— 範例輸出 —

— 範例輸入 —

— 範例輸出 —

— NOTE: —

TODO 競賽名稱

TODO 競賽名稱 試題本

---

附錄

TODO